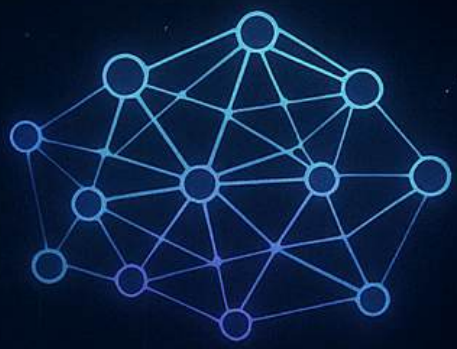
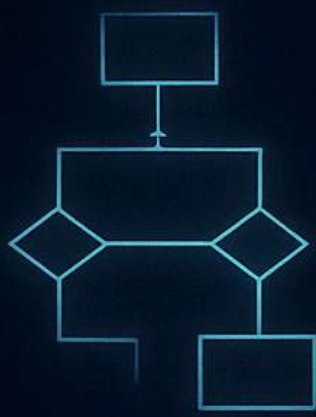
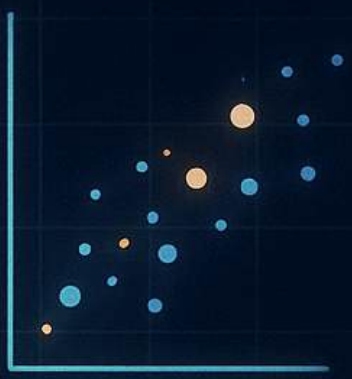
Seaborn

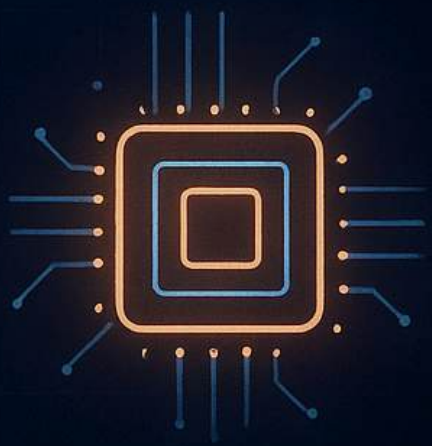
Arijit Das

```
import numpy as np
for i in range(len(data):
  resuit = processData(data[i])
def add(a, b):
  return a + b
```









Visualize Beyond the Data ✨

Data Visualization

SEABORN

What is Seaborn?

- Seaborn is a Python data visualization library built on top of Matplotlib.
- It provides many interactive and informative graphs and plots.
- It tightly integrated with Pandas, means you can directly use pandas DataFrame for visualization.

Why use Seaborn?

- less code, better visuals
- Built-in themes
- works directly with Pandas
- Provides specialized plots for statistical analysis

Seaborn	Matplotlib
• High level (easier, pre-built).	• low level (more control).
• Default is plain.	• Attractive by default.
• Less coding	• More coding.
• Works directly with Pandas DataFrame.	• Works with arrays.

Installation

```
pip install seaborn
```

- check installation

```
import seaborn as sns
print(sns.__version__)
```

- If you want to upgrade ↴

```
pip install --upgrade seaborn
```

Dependencies

Seaborn automatically installs ↴

- Matplotlib
- Pandas
- Scipy
- numpy

(A) Line Plot

What is Lineplot?

⇒ sns.lineplot() is a fonction that creates lines which are ideal for visualization relationships between two continuous variables. specially when you want to show trends over time.

(*) Syntax:

```
sns.lineplot(x = none, y = none, data = None)
plt.show()
```

(*) Key Parameters:

(i) hue: Groups data by colour (categorical).

(ii) size: Controls line thickness based on variables.

(iii) style: Groups data by linestyle (dashes, dots).

(iv) markers: ('o', '<').

(v) linestyles: ('-', '--', ':').

(B) Histogram plot

What is Histogram?

⇒ A chart that shows frequency distribution through bins in a dataset.

(*) Syntax:

```
sns.displot(df['col_name'], bins = [numeric datas], kde = True,
                                                    rug = True)
plt.grid(True)
plt.show()
```

Key parameters:

bins = Total number of bins.

Kde = Creates a smooth curve to show the distribution.

rug = To show the each data point.

color = you can set colors of the bars.

(c) Barplot:

What is Barplot?

⇒ A barchart shows the categorical information about any dataset.

* you can show the average, min, max, sum (total) of any category.

Syntax:

```
sns.barplot( x = 'category', y = 'numerical_data', data = data,
             estimator = None)
```

(i) Show Average:

```
sns.barplot( x = 'category', y = 'numerical_data', data = data,
             estimator = np.mean)
```

(ii) Show min:

```
sns.barplot( x = 'category', y = 'numerical_data', data = data,
             estimator = np.min)
```

(iii) Show max:

```
sns.barplot( x = 'category', y = 'numerical_data', data = data,
             estimator = np.max)
```

(iv) Show total:

```
sns.barplot( x = 'category', y = 'numerical_data', data = data,
             estimator = np.sum)
```

Key parameters:

hue = x-axis value / categorial data.

palette = 'Set2', the colors of each bars.

estimator = the aggregate function we want to perform.

x = categorical data.

y = Numerical data.

order = You can show all the categorical data in an order.

hue_order = Suppose you sub category under a category, you can order them also.

```
order = ['cat 1', 'cat 2', 'cat 3']

sns.barplot(x='cat', y='num', hue='cat', estimator=None,
            order=order, hue_order=[' ', ' '])
```

① Scatter plot:

What is Scatter plot?

⇒ sns.scatterplot() is a Seaborn function that helps to show the distribution of each category or numerical data.

Scatter plot Syntax:

```
sns.scatterplot(x='category', y='numerical_data', data=df,
                hue='cat')

sns.scatterplot(x='numerical_data', y='numerical_data', data=df,
                hue='cat')
```

Key parameters:

x = numerial / categorial data.

y = "

hue = It adds a categorical or numerie dimension by colouring different groups.

Think it like → " Group by this column and give each group a color

Heatmap

① What is heatmap?

⇒ `sns.heatmap()` is a seaborn function that represents the correlation between numerical variables.

Syntax:

```
sns.heatmap( corr , annot = True, linecolor = 'Black')
```

Step 1:

```
corr = df.corr()
```

It will work, if your dataset only contains numerical values, it will not calculate corr() if your dataset contains numerical + string object type values.

You have to drop those object type columns.

Step-2:

```
sns.heatmap(corr)
```

Key parameters:

corr = defines the correlation between two columns.

annot = This will show the each value of a column in a row.

linecolor = This will show linecolor between columns.

Count plot

① What is count plot?

⇒ `sns.countplot` is a Seaborn function that helps you to plot a graph, which automatically counts how many times each category appears. It calculates frequency of each category.

Example: "Male" appears 100 times & "Female" appears 80 times. Countplot will `count 100 vs 80`.

Syntax:

```
sns.countplot(x='categorical data', data=None, orient='v',
              order=None, hue=None, stat=None,
              hue_order=None)
```

Key Parameters:

(i) x = You only need a categorical column.
(ii) hue = The categorical column you want to group.
(iii) data = The dataset.
(iv) order = order of the category.
(v) hue_order = The sub categorical order.
(vi) stat = 'count' or 'Percentage' of the category.

Violin plot

What is violin plot?

⇒ Violin plot usually used for showing the distribution of the data.

Example:

suppose you are calculating sales data. Then you stock calculating the total sales of each day in a week, then you have to use the violin plot.

Syntax:

```
sns.violinplot(x='category', y='sale', data=None, hue=None,
               inner=None)
```

Key

(i) x = The categorical column.

(ii) y = The numerical column.

(iii) inner = 'quartile', 'box', 'point', shows individual data points.

(iv) data = The dataset / table.

(v) hue = The category we are choosing.

Box plot

What is box plot?

→ sns.boxplot in seaborn helps to visualize the box Quartile range (IQR) from a dataset.

Example:

Suppose you have sales dataset and you want to know the per day total percentage sales, the middle 50% sales, the maximum sales or the minimum sales. then you have to use boxplot.

Syntax:

sns.boxplot(x='cat', y='numeric', data= None, showmeans=True, hue='cat', palette = None linewidth=2)

Key features:

(i) x = Categorical data.
(ii) y = Numerical data.
(iii) hue = categorical data, which we want to calculate, based on that cat.
(iv) palette = The colour set.
(v) linewidth = The linewidth of each boxplot.
(vi) data = The dataset or table.
(vii) showmeans = The average of every box plots.

* Note: (If you want to show that categorical data horizontally)

sns.boxplot(x = "Numerical data", y = "categorial data", data = None)

* Just replace the categorical data with the numerical data.

Pair plot

What is Pair plot?

→ A pair plot in seaborn is a function that shows relationships between multiple features or numerical variables (columns).

Syntax:

```
sns.pairplot(data=df, hue=None, vars=[ ], kind='scatter',
             diag_kind='kde')

plt.show()
```

Key parameters:

i) data = The dataset
ii) hue = Adds color grouping by the given variables/columns.
iii) vars = The columns we want to find correlation.
iv) kind = 'scatter', 'kde', 'hist', 'reg'.

stripplot

What is stripplot?

→ `sns.stripplot()` in seaborn is a function which shows the each data points/values of a category.

Example:

Suppose you are analyzing a sales dataset, and you want to know each day sale, then you have to use stripplot().

Syntax:

```
sns.stripplot(x='categorical data', y='numerical data', hue='categorical
              palette='plasma', order=None, Jitter=True/False
```

Key parameters:

(i) x = you can put categorical / Numerical both type of data here.
(ii) y = you can put both (categorical / Numerical).
(iii) hue = Adds colour depending on the group.
(iv) palette = colours for each category.
(v) order = You can change order based on the category.
(vi) jitter = [True / False]. (Maintain gaps between datapoints).
(vii) Colour = colours for each category.

Catplot

What is catplot?

→ Catplot is like a short cut method - instead of creating many categorics plot manually, we can use catplot.

Example:

It depends on you, what type of plots or analysis you want to do.

(i) Box plot ⟶ If you want to know the outliers or 50% middle value.
(ii) Barchart ⟶ If you want to know the (Sum, mean, min, max) of any category
(iii) Violinplot ⟶ If you want to know the distribution of the data.

Syntax:

```
sns.catplot(x='category', y='numerical data', data=None, kind=None,
            hue=None, palette=None, col=None, order=None)
```

Key parameters:

X= The categorical data.

Y= The numerical data.

data= The dataset.

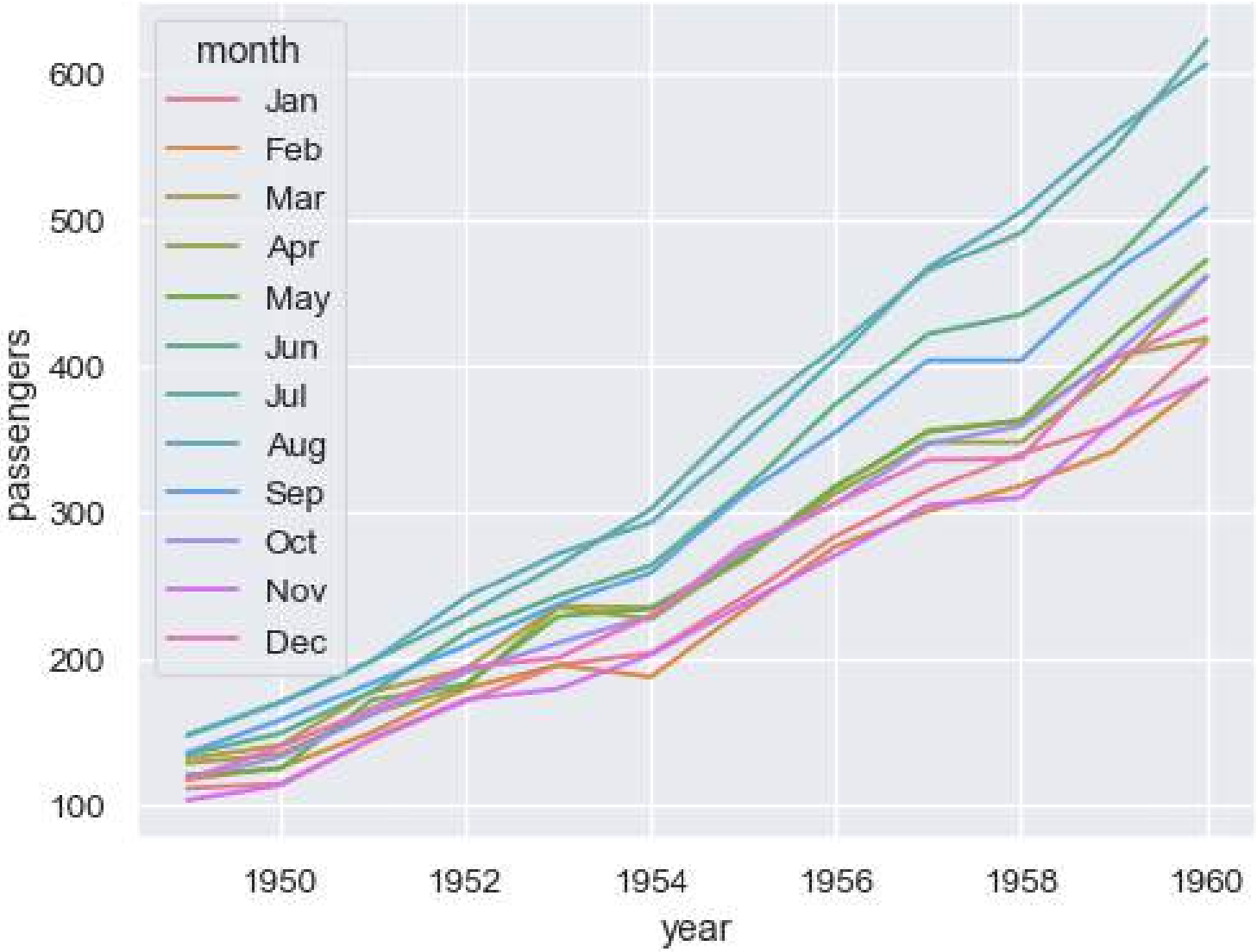
hue = The colours grouping based on categories.

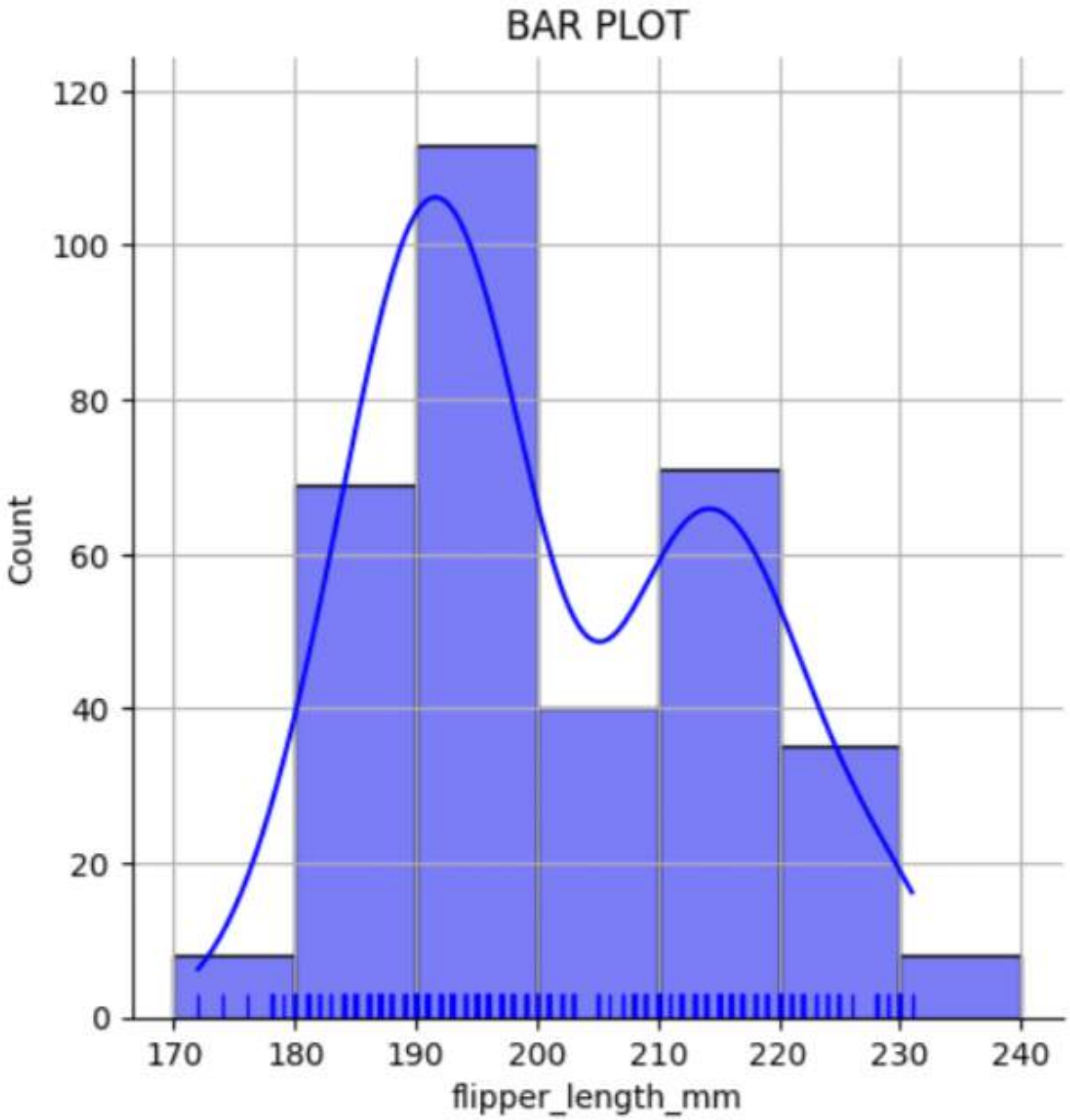
Col = It is used for faceting — spliting the datasets based on categorical variable.

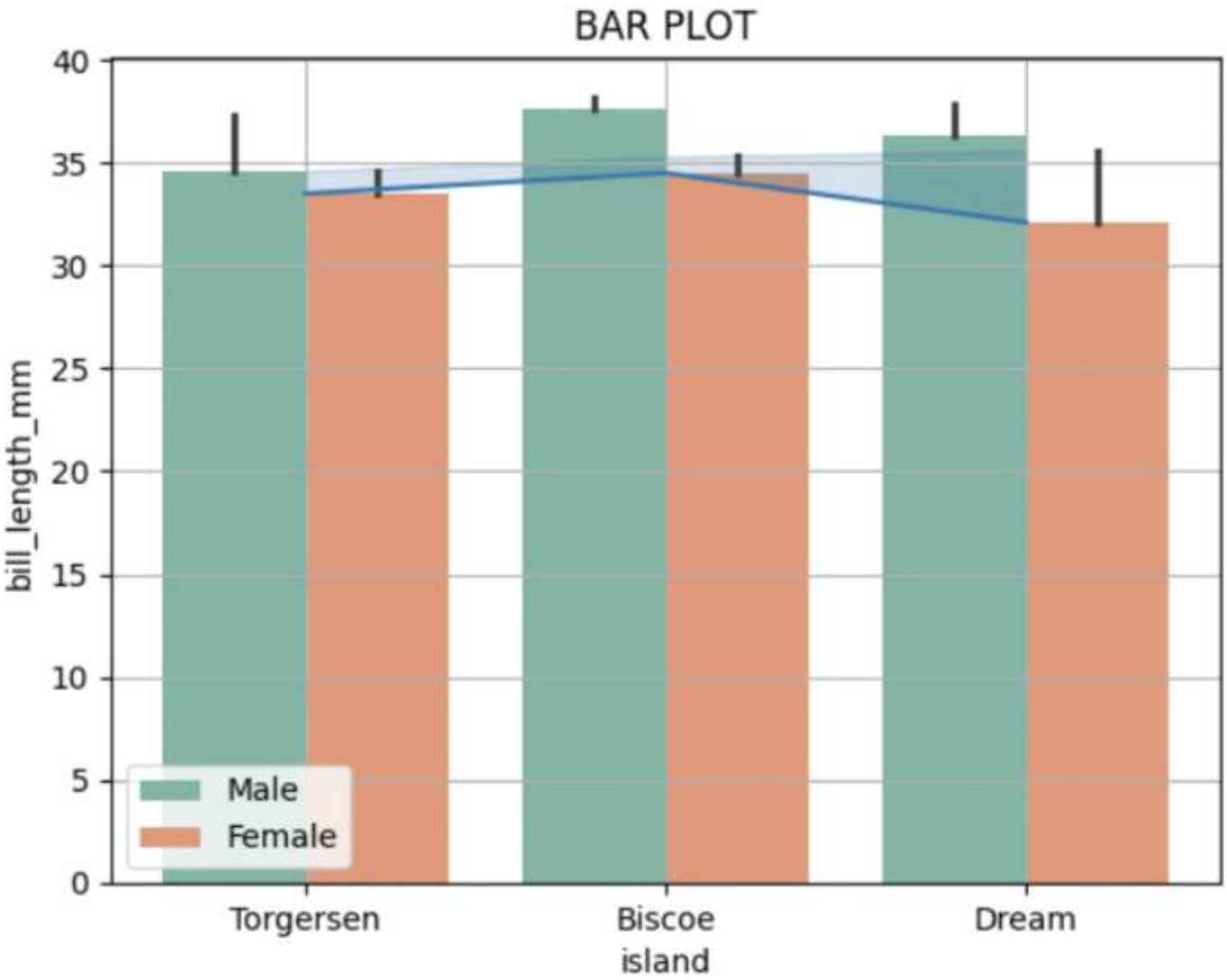
Order= used to control the order of the categorical values.

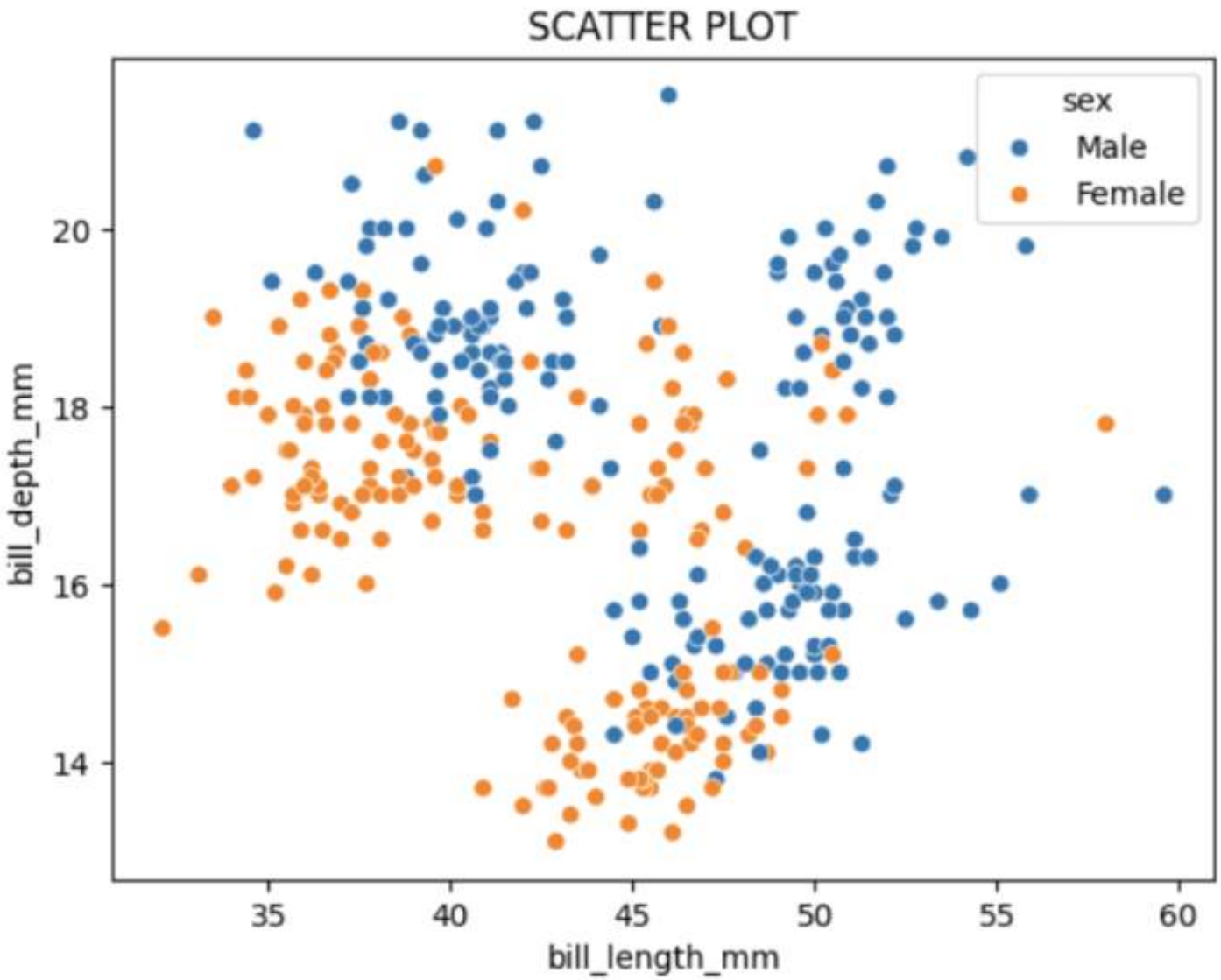
Kind= [Boxplot, barplot, violinplot]

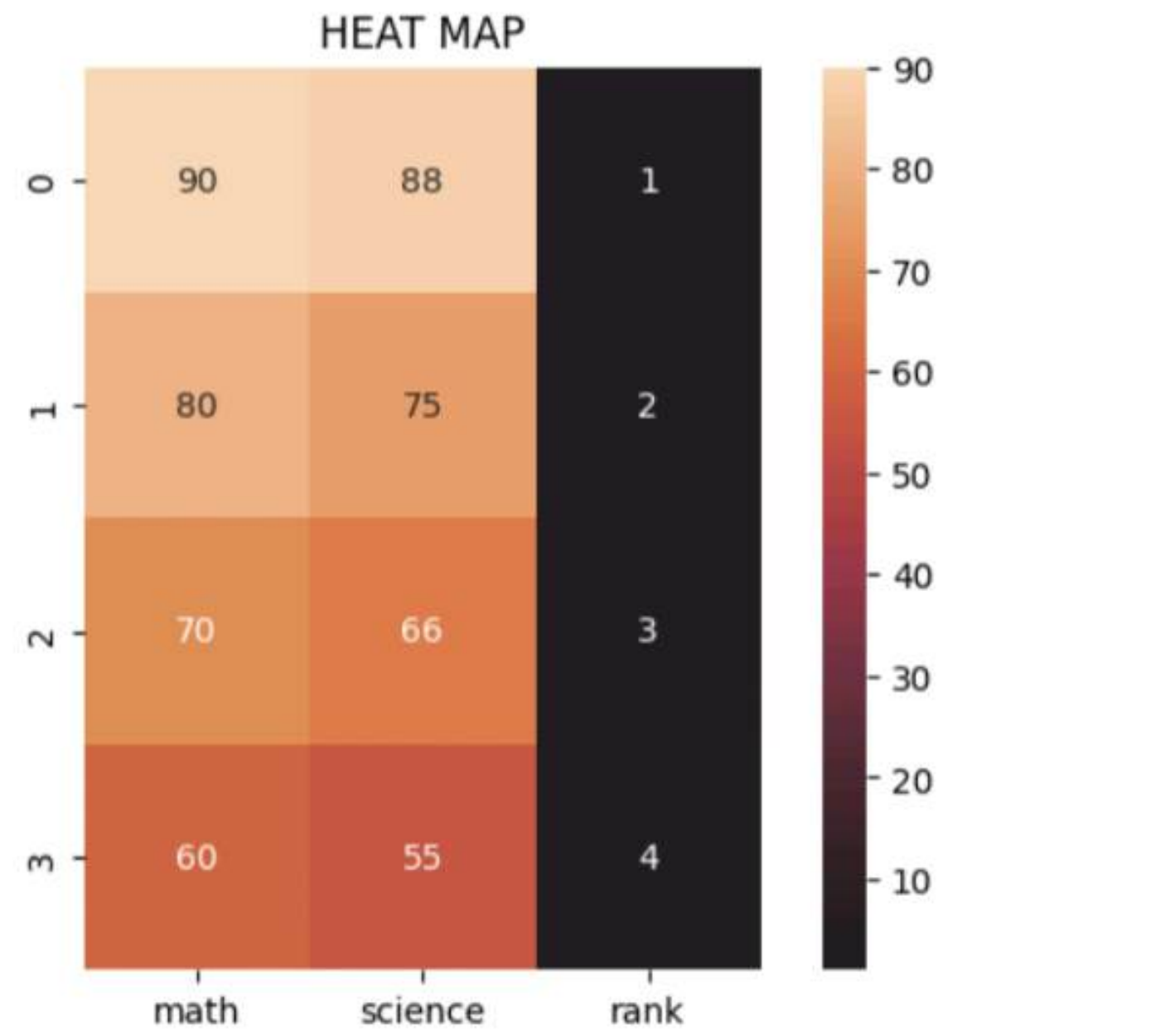
estimator= [np.sum, np.min, np.max, np.mean]

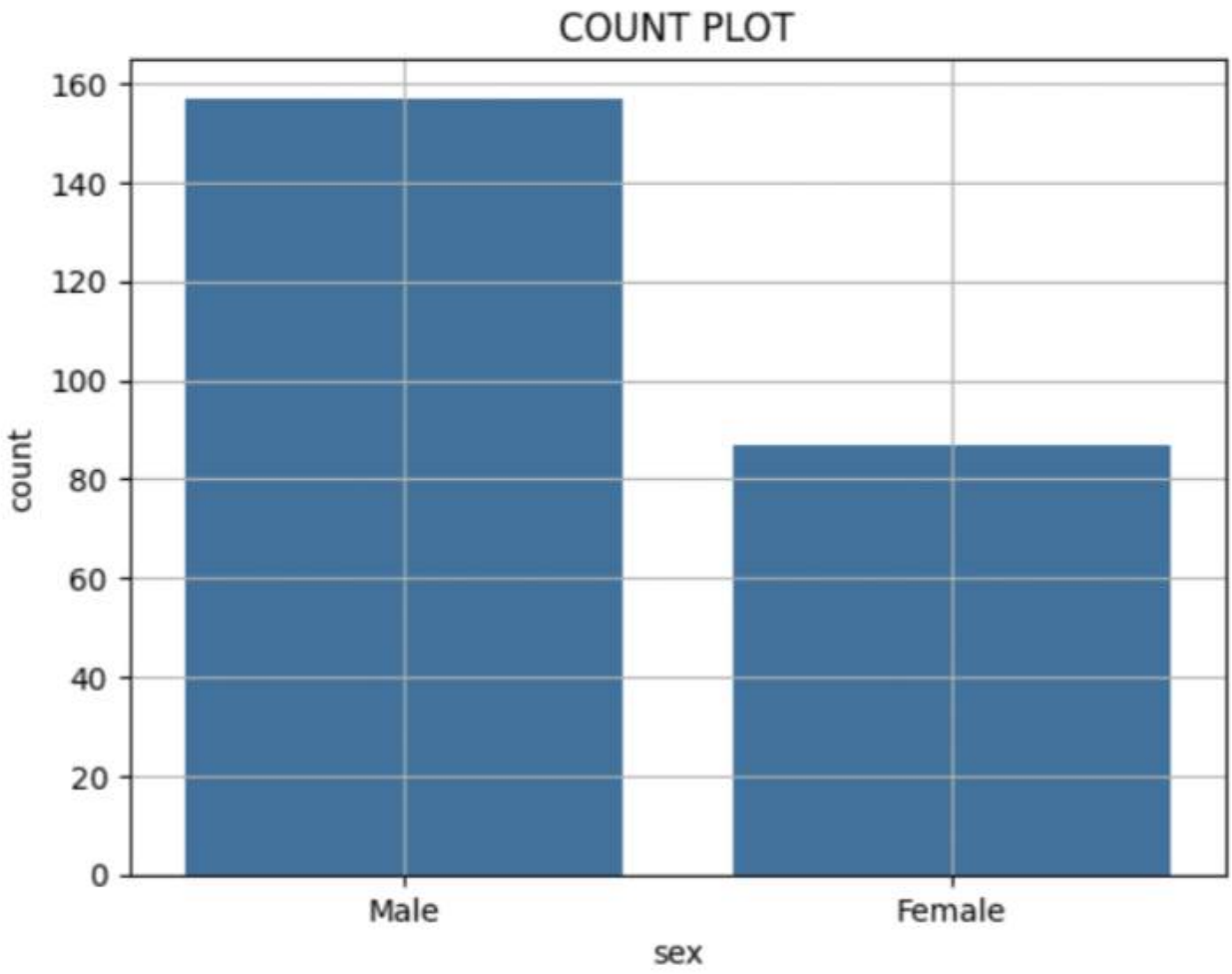

month
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
600
500
400
300
200
100
passengers
1950
1952
1954
1956
1958
1960
year

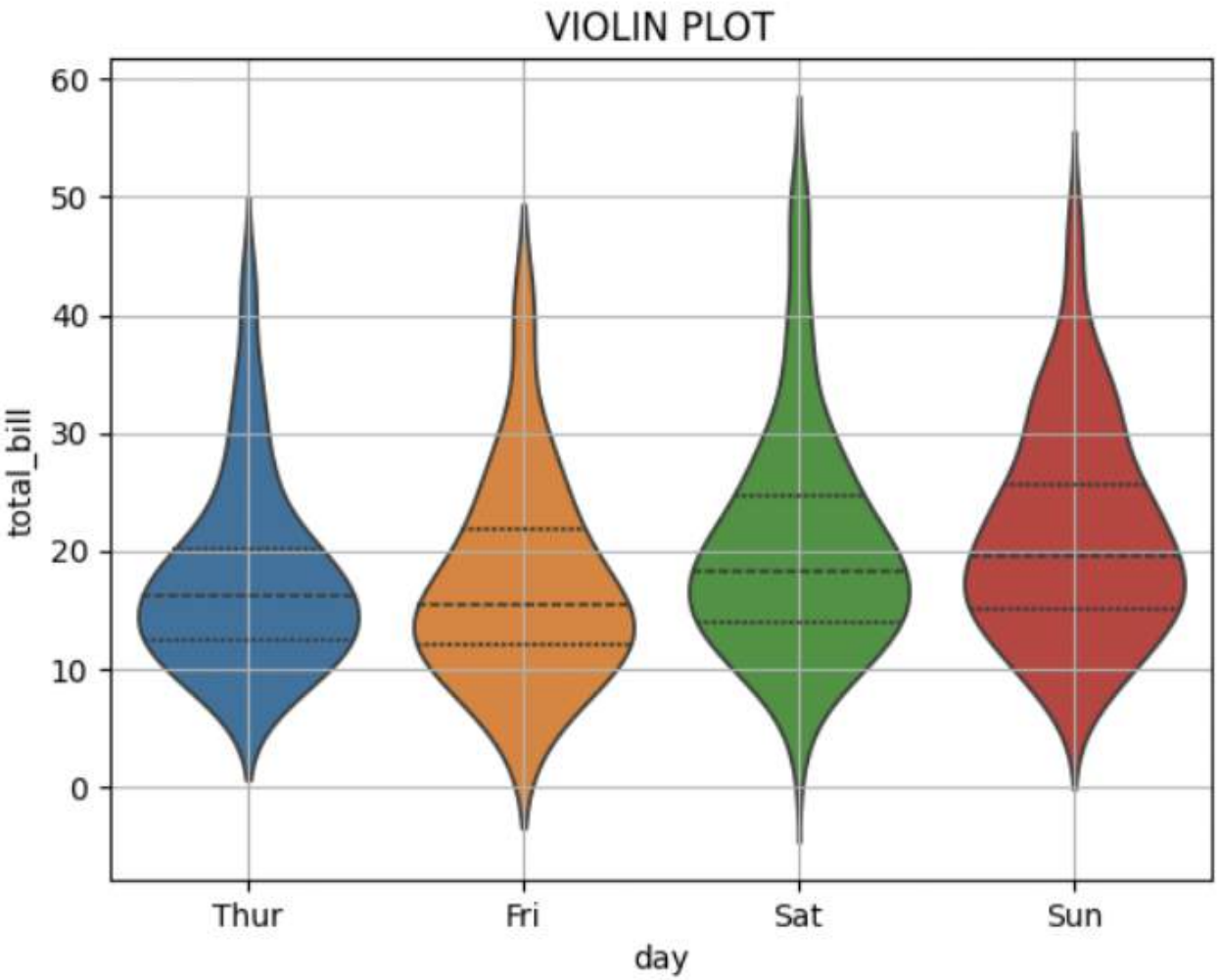

BAR PLOT
120
100
80
60
40
20
0
Count
170
180
190
200
210
220
230
240
flipper_length_mm

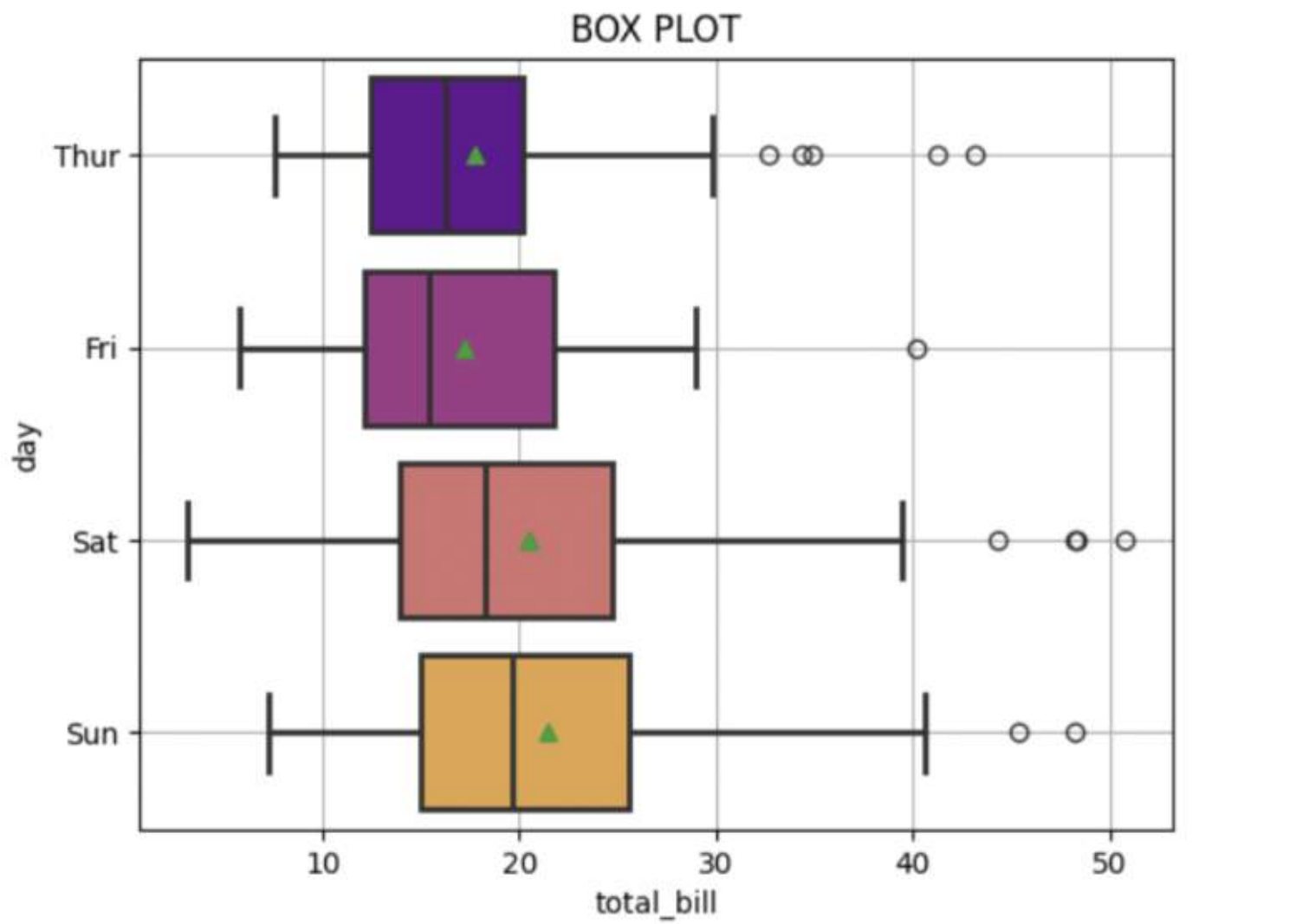

BAR PLOT
40
35
30
25
20
15
10
5
0
bill_length_mm
Male
Female
Torgersen
Biscoe
Dream
island

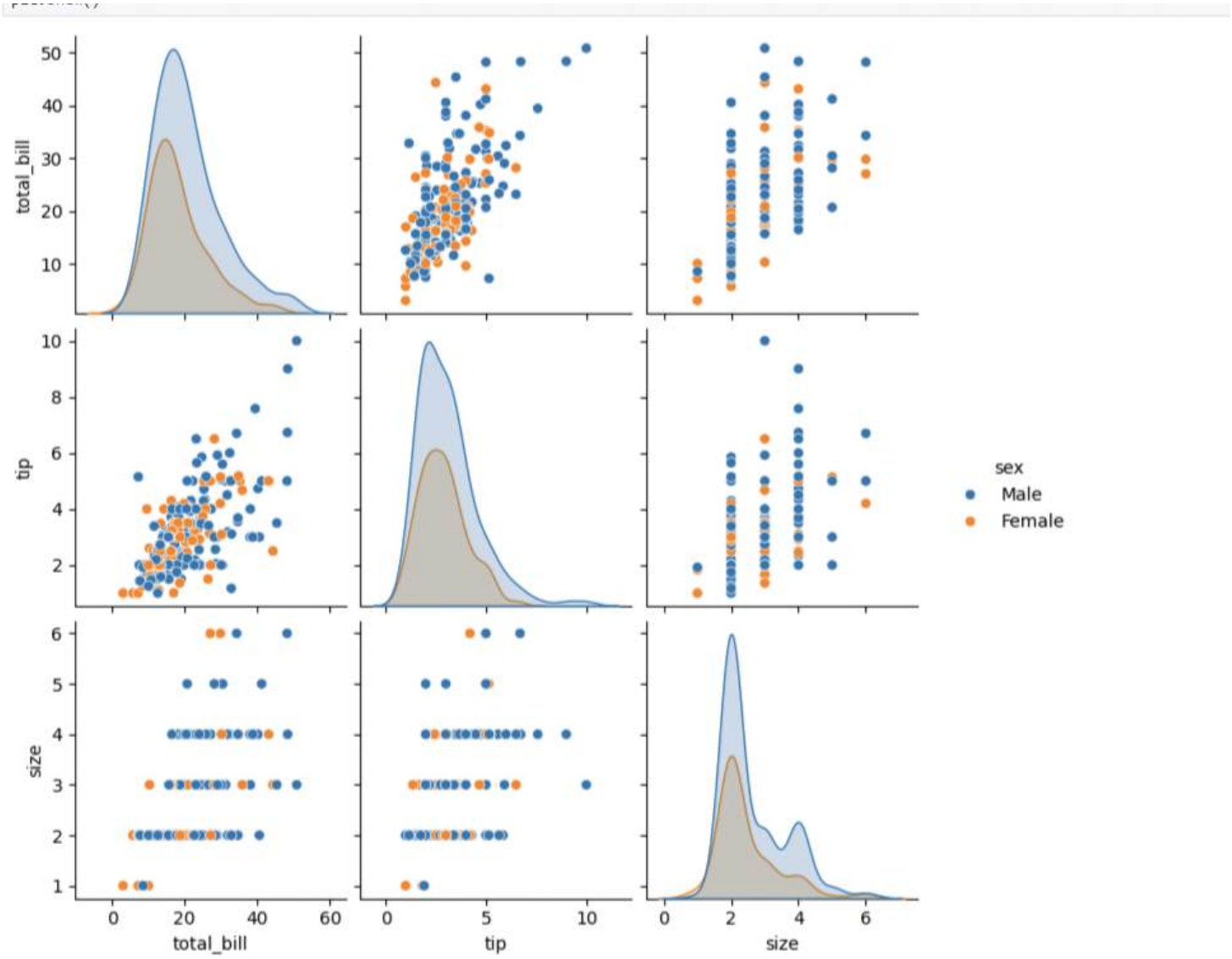

SCATTER PLOT
sex
Male
Female
bill_depth_mm
20
18
16
14
35
40
45
50
55
60
bill_length_mm

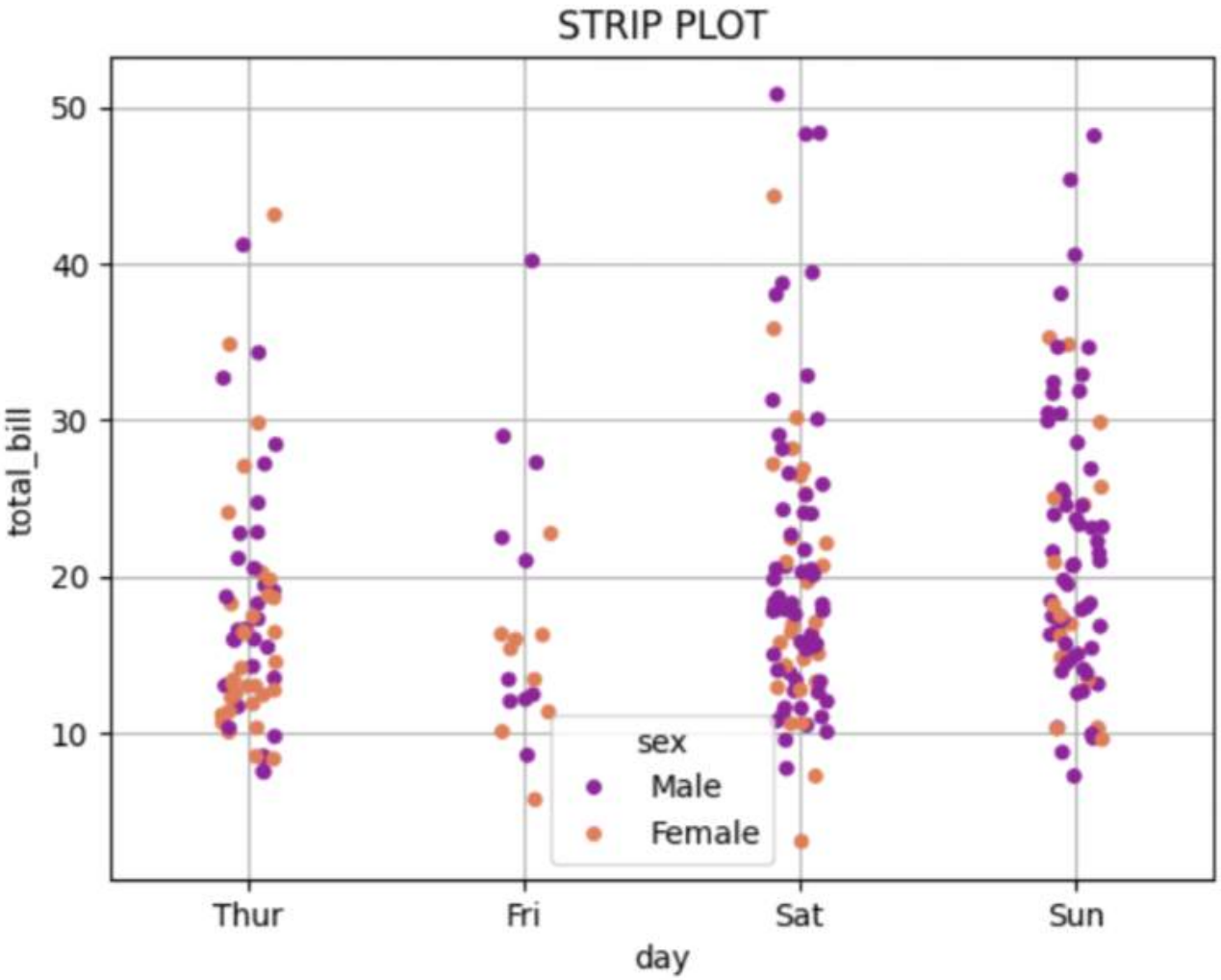

HEAT MAP
0
1
2
3
90
88
1
80
75
2
70
66
3
60
55
4
math
science
rank
90
80
70
60
50
40
30
20
10

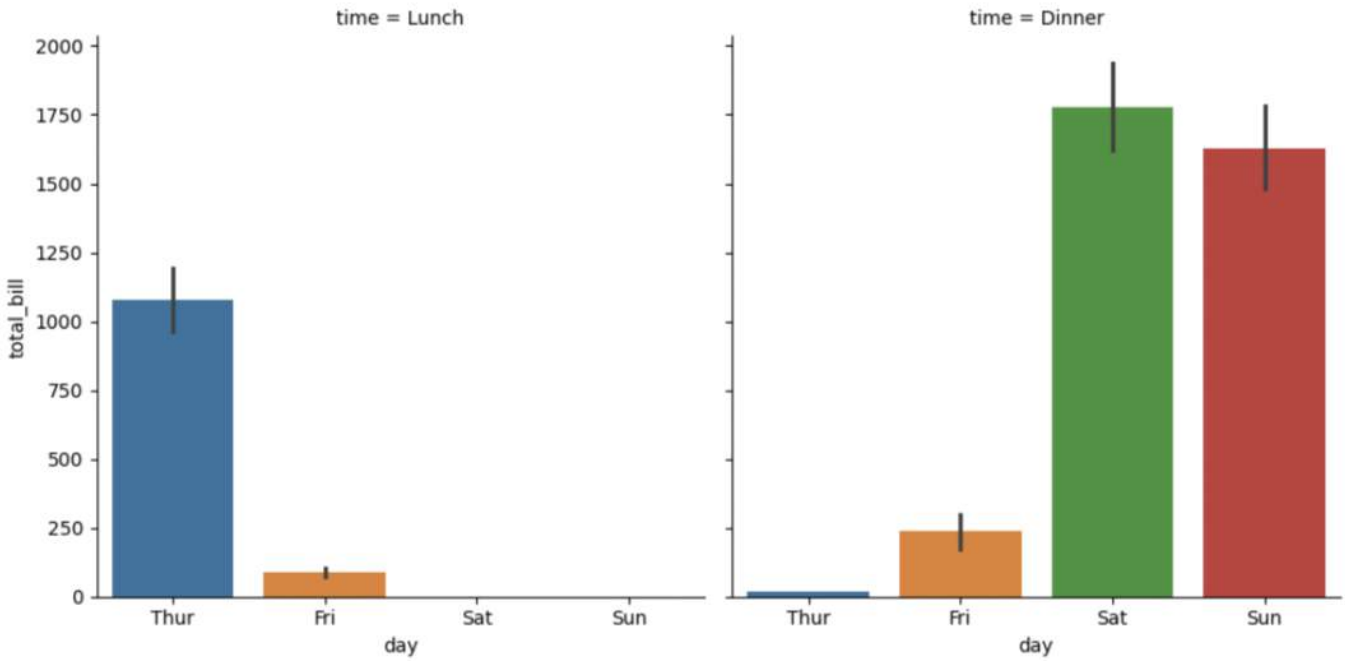

COUNT PLOT
160
140
120
100
80
60
40
20
0
count
Male
Female
sex


VIOLIN PLOT
60
50
40
30
20
10
0
total_bill
Thur
Fri
Sat
Sun
day


BOX PLOT
Thur
Fri
Sat
Sun
day
10
20
30
40
50
total_bill


total_bill
tip
size
sex
Male
Female


STRIP PLOT
total_bill
50
40
30
20
10
sex
Male
Female
Thur
Fri
Sat
Sun
day


time = Lunch
time = Dinner
total_bill
2000
1750
1500
1250
1000
750
500
250
0
Thur
Fri
Sat
Sun
day